

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 2 : Faire obéir le programme (Making the Sketch Do Your Bidding)

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

2.0 Introduction au Chapitre 2

Maîtriser le code Arduino

Ce chapitre couvre les concepts et tâches logiciels essentiels pour programmer l'Arduino[cite : 5]. Bien que basé sur le C/C++, l'Arduino masque une partie de la complexité pour rendre le code plus accessible.

Objectifs de cette section :

- Comprendre les types de variables et la gestion de la mémoire.
- Manipuler des groupes de valeurs (Tableaux) et du texte (Strings).
- Contrôler le flux du programme (Conditions, Boucles).
- Structurer son code avec des fonctions personnalisées[cite : 9].

2.1 Structurer un programme Arduino

Problème : Comment un programme Arduino s'exécute-t-il dans le temps ?

Solution : Le flux d'exécution se divise en trois zones temporelles :

1. **Déclarations globales :** Avant le `setup()`. Les variables déclarées ici existent pendant toute la durée de vie du programme.
2. `setup()` : Exécuté une seule fois après la mise sous tension ou un Reset.
3. `loop()` : La boucle principale. Dès qu'elle se termine, elle recommence instantanément.

Figure 1: *Schéma conceptuel du flux d'exécution Arduino (Insérez image ici)*

2.2 Les types primitifs simples (Variables)

Les variables sont des "boîtes" en mémoire. Le C++ est un langage *fortement typé* ; chaque variable doit avoir un type défini[cite : 5].

- `boolean` : Prend la valeur `true` ou `false` (1 octet).
- `char` : Stocke un caractère ASCII, ex : `'A'` (1 octet).
- `byte` : Entier non signé de 0 à 255 (1 octet).
- `int` : Entier signé de -32 768 à 32 767 (2 octets sur Uno).
- `long` : Grand entier de -2 147 483 648 à 2 147 483 647 (4 octets).

Attention aux dépassements (Overflow)

Si vous ajoutez 1 à un `int` valant 32 767, il bascule à -32 768 de manière imprévisible !

2.3 Utilisation des nombres à virgule (Floating-Point)

Problème : Vous avez besoin de nombres décimaux (ex : température 23.5° C).

Solution : Utiliser les types float ou double[cite : 23].

```
float temperature = 23.5;  
float precision_calcul = 10.0 / 3.0; // Donne 3.333333
```

Contraintes importantes :

- Sur l'Arduino Uno, double a la même taille que float (32 bits), contrairement aux PC standards[cite : 23].
- Les calculs en float sont **très lents** car l'Arduino (ATmega328) n'a pas de processeur mathématique dédié (FPU).
- **Précision :** Un float n'est précis qu'à 6 ou 7 chiffres significatifs.

2.4 Travailler avec des groupes de valeurs (Tableaux)

Problème : Vous devez stocker 10 relevés de capteurs sans créer 10 variables.

Solution : Utiliser un tableau (*Array*).

```
int capteurs[5]; // Declare un tableau de 5 entiers
int broches[] = {2, 4, 8, 3, 6}; // Declare et initialise en meme temps

void setup() {
    // Les index commencent TOUJOURS a 0 !
    int premiereBroche = broches[0]; // Vaut 2
    broches[4] = 9; // Remplace le 6 par un 9
}
```

Danger des Tableaux en C++

Si vous écrivez dans `broches[5]`, le compilateur ne vous arrêtera pas. Vous écrirez dans une autre partie de la mémoire, causant des bugs mystérieux (Memory Corruption).

2.5 Les Chaînes de caractères (Arduino String Object)

Problème : Manipuler du texte facilement.

Solution : Utiliser la classe String d'Arduino[cite : 8].

```
String message = "Bonjour !";
String nom = "Dr DIOP";

void setup() {
  // Concat nation tres simple avec '+'
  String complet = message + " Je suis " + nom;

  // Fonctions utiles :
  int longueur = complet.length();
  complet.toUpperCase(); // Convertit en majuscules
}
```

Note : Pratique, mais consomme beaucoup de mémoire RAM (fragmentation dynamique)[cite : 8]. À éviter sur les gros projets.

2.6 Les Chaînes C traditionnelles (Character Strings)

Problème : L'objet String sature la petite RAM (2ko) de l'Uno.

Solution : Utiliser des tableaux de caractères bruts (*C Character Strings*)[cite : 8].

```
// Le compilateur ajoute automatiquement le caractere nul '\0' a la fin
char chaineA[8] = "Arduino"; // 7 lettres + '\0' = 8 octets

// Utilisation de fonctions C standard :
char buffer[20];
strcpy(buffer, "Capteur: "); // Copie
strcat(buffer, "12");         // Ajoute a la suite (Capteur: 12)
int len = strlen(buffer);     // Donne la longueur (11)
```

Figure 2: Représentation en mémoire d'une chaîne terminée par un caractère nul '\0'

2.8 et 2.9 Conversions : Nombres et Strings

Conversion String vers Nombre[cite : 9] :

```
// Methode 1 : Avec la classe String
String chaineNombre = "1234";
int valeur = chaineNombre.toInt(); // valeur vaut 1234

// Methode 2 : Avec les chaines C et atoi()
char buffer[5] = "5678";
int valeurC = atoi(buffer); // ASCII to Integer
```

Lire depuis le Moniteur Série : Depuis Arduino 1.0, la fonction `parseInt()` extrait directement des nombres depuis le port série[cite : 9].

```
int delaiClignotement = Serial.parseInt(); // Attend un entier
```

2.10 Structurer le code en fonctions (Functional Blocks)

Problème : Votre `loop()` fait 500 lignes et devient illisible.

Solution : Créer des fonctions personnalisées[cite : 9].

```
// Fonction qui prend un parametre et ne renvoie rien (void)
void clignoter(int temps) {
    digitalWrite(13, HIGH);
    delay(temps);
    digitalWrite(13, LOW);
    delay(temps);
}

void loop() {
    clignoter(500); // Code principal tres clair et lisible !
}
```

Avantage : Facilite la maintenance et permet la réutilisation du code ("Don't Repeat Yourself")[cite : 9].

2.11 Retourner des valeurs depuis une fonction

Problème : Une fonction doit calculer et renvoyer un résultat.

Solution : Utiliser le mot-clé `return` avec le type approprié.

```
// Cette fonction renvoie un 'int'
int multiplierParDeux(int valeur) {
    int resultat = valeur * 2;
    return resultat;
}

void loop() {
    int x = multiplierParDeux(10); // x vaudra 20
}
```

Retourner PLUSIEURS valeurs ?

En C++, on ne peut retourner qu'une seule valeur via `return`. Pour en renvoyer plusieurs, il faut passer des variables "par référence" (en utilisant des pointeurs `&`), afin que la fonction modifie directement la mémoire d'origine.

2.12 et 2.16 Prendre des décisions conditionnelles

Problème : Exécuter du code uniquement si un capteur dépasse un seuil.

```
int temperature = analogRead(A0);

// Structure if / else standard
if (temperature > 500) {
    digitalWrite(alarmePin, HIGH);
} else if (temperature < 100) {
    digitalWrite(chauffagePin, HIGH);
} else {
    // Si tout est normal
    digitalWrite(alarmePin, LOW);
}
```

Le Switch Case : Idéal pour analyser un menu série (caractères uniques).

```
switch(lettreRecue) {
    case 'A': avancer(); break;
    case 'R': reculer(); break;
    default: stopper(); break;
}
```

2.13 et 2.14 Les Boucles (Loops et Counters)

La boucle while (Tant que) : S'exécute tant que la condition est vraie.

```
int x = 0;
while (x < 5) {
    Serial.println(x);
    x++; // Ne pas oublier d'incrémenter, sinon boucle infinie !
}
```

La boucle for (Pour) : Structure condensée, parfaite pour les tableaux.

```
// Structure : for(initialisation; condition; incrementation)
for (int i = 0; i < 5; i++) {
    pinMode(broches[i], OUTPUT); // Configure 5 broches d'un coup
}
```

Le mot-clé `break` permet de "casser" (sortir prématurément) de n'importe quelle boucle.

2.19 et 2.20 Opérations logiques et binaires

Comparaisons Logiques (Booléennes) :

- **ET (AND)** : `if (temp > 20 && lumiere > 50)`
- **OU (OR)** : `if (bouton == HIGH || alarme == HIGH)`
- **NON (NOT)** : `if (!capteurActif)`

Opérations "Bitwise" (Niveau binaire) : Utilisées pour manipuler directement les registres du microcontrôleur (très rapide).

- `a & b` : ET bit à bit.
- `a | b` : OU bit à bit.
- `a ^ b` : OU Exclusif (XOR).
- `a << 2` : Décalage à gauche (multiplie par 4).

Variables locales vs globales

Si vous déclarez une variable `int x`; à l'intérieur d'une fonction ou d'un `if`, elle n'existe **que** dans ce bloc (Portée locale). Le `loop()` ne pourra pas la voir !

Erreurs de syntaxe mortelles en C++ :

- **Assignment vs Comparaison :**
 - `if (x = 5)` : Ceci met 5 dans x, et la condition est toujours vraie !
 - `if (x == 5)` : Ceci est la correcte comparaison.
- Oublier le point-virgule ; à la fin de l'instruction.
- Dépassement d'index dans un tableau (cause 90% des redémarrages inexplicables du microcontrôleur).

- **Types** : Choisissez toujours le type le plus petit possible (byte au lieu de int) pour économiser la mémoire de l'Arduino.
- **Strings vs char []** : Préférez les tableaux de caractères en C pour des projets stables sur de longues durées[cite : 8].
- **Fonctions** : Organisez votre logique en sous-routines claires[cite : 9]. Un loop() bien conçu ne devrait contenir que des appels de fonctions qui se lisent comme un livre.

Fin du Chapitre 2. Le prochain chapitre abordera les "Opérations Mathématiques", le calcul de probabilités et l'utilisation de capteurs paramétrés.